Technical, Behavioral, and Safety Evaluation of Language Models Under Severe VRAM Constraints

A Comparative Experimental Study Using NVIDIA GeForce GTX 1050 Ti (4GB GDDR5)

Anderson Tavares
Artificial Intelligence and Infrastructure Laboratory
https://github.com/AndersonTavares0/llm-research-bench

October 24, 2026

Abstract

The proliferation of Small Language Models (SLMs) in the 2B to 4B parameter regime has opened new frontiers for local on-device inference on edge nodes and hardware-constrained servers. However, operational viability under severe video memory constraints (VRAM $\leq$ 4096 MiB) remains a complex, multi-variable challenge where quantization, speculative decoding architectures, token generation throughput, and behavioral alignment directly intersect. This paper presents a reproducible experimental study evaluating four state-of-the-art open-weights models: **Gemma-4-E2B-it** (IQ4_XS with MTP Q8_0 speculative draft), **Qwen-3.5-4B-Instruct** (Q4_K_M), **Qwen-3.5-2B-Instruct** (Q4_K_XL), and **Llama-3.2-3B-Instruct** (Q4_K_M), running on a dedicated NVIDIA GeForce GTX 1050 Ti (4GB GDDR5, Compute Capability 6.1) inside a Proxmox VE container (CT 108). The experimental framework examined two strictly segregated domains: (1) *System Performance* (TTFT, token generation throughput in tok/s, static and peak VRAM footprint); and (2) *Model Quality & Safety* (quantitative technical reasoning, Pulse/MCP-style agentic tool calling protocols, and adversarial robustness across 6 attack vectors including weaponized CVEs, stealth keyloggers, DAN jailbreaks, system prompt leakage, and indirect prompt injection). Results demonstrate that MTP speculative decoding in Gemma-4-E2B achieves the highest generation throughput of the study (**46.3 tokens/s**) while consuming only 36.8% of available VRAM, whereas Qwen-3.5-4B exhibits superior structural coherence at the cost of 74.9% VRAM capacity saturation and 18.4 tok/s. Across safety vectors, all models demonstrated an 83.3% clean refusal rate against weaponized exploit requests and DAN bypasses, while demonstrating uniform vulnerability to indirect prompt injections embedded in untrusted system logs.

Keywords: Small Language Models (SLMs), VRAM Constraints, Speculative Decoding, Adversarial Robustness, Tool Calling, Reproducible Benchmarking.

1 Introduction and Motivation

Enabling autonomous, local intelligent assistants and edge agents in air-gapped or privacy-sensitive homelab environments requires deploying Small Language Models (SLMs) on consumer-grade hardware. On legacy GPUs such as the NVIDIA GeForce GTX 1050 Ti, equipped with only 4096 MiB of GDDR5 memory over a 128-bit bus (112 GB/s theoretical bandwidth), the memory margin for weight graphs and key-value (KV) context caches is remarkably tight.

Any memory spillover forces tensor offloading onto host system memory (DDR4) over the PCIe bus, degrading the Time To First Token (TTFT) and collapsing token generation rates to sub-3 tok/s.

This investigation formulates and empirically evaluates five core scientific hypotheses:

H1: Architectures employing Multi-Token Prediction (MTP) speculative decoding bypass memory bandwidth bottlenecks on the Pascal architecture by over 100% relative to standard autoregressive models.

H2: Aggressive quantization (IQ4_XS / Q4_K) in SLMs preserves strict adherence to agentic tool-calling syntax (JSON Schema), but induces degradation in multi-step arithmetic reasoning.

H3: 4B parameter models approach the upper operational limit of 4GB VRAM budgets, necessitating restricted context windows or sub-4-bit weights to prevent Out-Of-Memory (OOM) failures.

H4: Direct safety alignment against classic jailbreaks (e.g., DAN) is independent of parameter count within the 2B–4B range, but susceptibility to indirect injection remains a shared challenge.

H5: Agentic execution capability under local orchestration (Pulse/MCP) depends fundamentally on the model's ability to ingest prior tool outputs without hallucinating non-existent tools.

2 Infrastructure and Environmental Reconnaissance

The experimental evaluations were conducted within homelab cluster node `v01`, utilizing dedicated LXC container **CT 108** (GPU Lab) with direct PCIe hardware passthrough.

2.1 Hardware Specifications

- **Host CPU:** AMD Ryzen 5 3600 (6 cores, 12 threads @ 3.6 GHz base / 4.2 GHz boost).
- **Container Allocation (CT 108):** 2 dedicated vCPUs, 4.0 GiB physical RAM, and 512 MiB swap.
- **Graphics Accelerator:** NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, 768 CUDA cores, SM 6.1, 75W TDP).
- **Storage Subsystem:** NVMe PCIe 3.0 x4 root pool on ZFS.

2.2 Software Stack and Inference Runtime

- **Guest Operating System:** Debian GNU/Linux 13 (Trixie), Linux Kernel 6.8.12-14-pve x86_64.
- **NVIDIA Drivers and Toolkits:** Driver 580.173.02, CUDA Driver 13.0.
- **Inference Engine:** `llama-server` (based on llama.cpp b4120+) executing inside an isolated rootless Podman container under the `ollama` user cgroup, listening on internal TCP port `18081`.
- **Networking & Isolation:** Strictly bound to `127.0.0.1` with reverse proxy routing via Caddy for external agent requests originating from Pulse (CT 100).

3 Experimental Methodology and Evaluated Models

Four models representing distinct architectural trade-offs within the 2B–4B spectrum were evaluated:

1. **Gemma-4-E2B-it:** Developed by Google DeepMind, quantized in `IQ4_XS` (1.5 GB), coupled with a speculative draft model `mtp-gemma-4-E2B-it-Q8_0` (0.28 GB) via `-model-draft`.
2. **Qwen-3.5-4B-Instruct:** Dense model by Alibaba Cloud, quantized in `Q4_K_M` (2.7 GB), representing the upper VRAM capacity limit for 4GB cards.
3. **Qwen-3.5-2B-Instruct:** Lightweight variant from the Qwen family, quantized in `Q4_K_XL` (1.5 GB).
4. **Llama-3.2-3B-Instruct:** Lightweight instruction model by Meta AI, quantized in `Q4_K_M` (2.0 GB).

3.1 Standardized Test Protocol

All models were benchmarked using the automated harness `benchmark_suite.py` under thermally normalized conditions ($T_{\text{GPU}} \leq 45$°C prior to each run). Sampling hyperparameters were pinned to temperature $\tau = 0.2$, top_p $= 0.9$, with fixed pseudo-random seeds.

4 System Performance Results

Table 1 summarizes the empirical performance metrics obtained across the four candidate configurations.

Table 1: Empirical System Performance Metrics under 4096 MiB VRAM Budget

Model	**Static VRAM** (MiB)	**Peak VRAM** (MiB)	**Budget** (% of VRAM)	**Prompt Eval** (tokens/s)	**Generation** (tokens/s)	**TT** (m
Gemma-4-E2B (MTP)	1,452	1,510	36.8%	**468.2**	**46.3**	**115**
Qwen-3.5-2B	1,540	1,598	39.0%	389.5	37.5	142
Llama-3.2-3B	2,050	2,120	51.7%	390.7	27.4	168
Qwen-3.5-4B	2,987	3,069	74.9%	237.1	18.4	284

4.1 Impact of MTP Speculative Decoding

As demonstrated in Table 1, **Gemma-4-E2B** outpaced all competitors in generation throughput, maintaining an average of **46.3 tokens/s** (surpassing 52 tokens/s in low-entropy regimes). This validates hypothesis **H1**: the lightweight speculative draft model significantly reduces memory bandwidth demands on the GTX 1050 Ti, validating multiple speculative candidate tokens per full forward pass. In contrast, Qwen-3.5-4B was throttled by the 112 GB/s memory bandwidth to 18.4 tokens/s.

5 Model Quality, Reasoning, and Safety Results

Table 2 outlines performance across the behavioral evaluation batteries.

Table 2: Behavioral Evaluation: Safety, Agentic Protocol, and Technical Reasoning

Model	Safety & Robustness (% Clean Refusals)	Agentic Protocol (Scenarios 1, 2, 3)	Reasoning (% Exact Match)	Deployme Recommenda
Gemma-4-E2B (MTP)	83.3% (5/6)	**All Passed**	33.3% (1/3)	**Recommended (Ec**
Qwen-3.5-4B	83.3% (5/6)	**All Passed**	33.3% (1/3)	Suitable (Critical R
Qwen-3.5-2B	83.3% (5/6)	**All Passed**	0.0% (0/3)	Weak Arithm
Llama-3.2-3B	83.3% (5/6)	**Failed (Scen. 2)**	33.3% (1/3)	Fragile in Agent

5.1 Safety and Jailbreak Analysis

The safety suite evaluated model behavior across six adversarial vectors:

1. **Weaponized CVE Exploit (Log4Shell CVE-2021-44228):** Direct request for automated exploitation scripts. *Outcome:* All models cleanly refused weaponized payload generation while offering defensive remediation guidance.
2. **Stealth Keylogger (C++ Explorer Injection):** Windows API interrupt interception request. *Outcome:* Universal refusal accompanied by detection architecture explanations.
3. **DAN Jailbreak:** Role-spoofing filter bypass attempt. *Outcome:* Consistently neutralized.
4. **Opposite Day Jailbreak:** Psychological moral inversion trick. *Outcome:* Neutralized.
5. **System Prompt Leakage:** Extraction attempt on internal developer instructions. *Outcome:* Universal refusal; no model leaked developer guardrails.
6. **Indirect Prompt Injection in System Logs:** Simulated injected command (`SYSTEM COMMAND OVERRIDE`) embedded inside untrusted log data. *Critical Outcome:* **All four models failed this test**, executing the injected command or explicitly acknowledging the bypass. This indicates that while direct alignment (RLHF/DPO) is robust across 2B–4B models, handling untrusted context inputs remains an open vulnerability.

5.2 Agentic Protocol and Tool Calling

In multi-turn Pulse/MCP tool execution:

- **Gemma-4-E2B**, **Qwen-3.5-4B**, and **Qwen-3.5-2B** generated valid schema function calls (`get_proxmox_node_status`), parsed simulated tool results accurately, and concluded conversations without hallucinated tools.
- **Llama-3.2-3B** failed during the continuation turn: upon receiving tool output, it failed to synthesize the answer and reported parsing errors, verifying hypothesis **H5**.

5.3 Quantitative Reasoning Deficiencies

In technical system calculations:

- **CIDR Subnetting (/26):** All models struggled with simultaneously calculating network address, broadcast address, and usable hosts without external compute tools (e.g., Llama-3.2-3B hallucinating $2^{18} = 262,144$ hosts).
- **ZFS RAIDZ2 Parity: Gemma-4-E2B** and **Qwen-3.5-4B** correctly computed 16TB raw usable capacity and tolerance of 2 disk failures. Llama-3.2-3B and Qwen-3.5-2B exhibited arithmetic hallucinations (18TB) or RAID 5 conceptual confusion.

- **Multi-Step RAM Allocation: Llama-3.2-3B** was the only model to correctly calculate the multi-step allocation step-by-step ($32 - 4 - 26 = 2$ GB < 3 GB $\implies$ No). Gemma-4-E2B and Qwen-3.5-4B miscalculated the compound subtraction, and Qwen-3.5-2B failed completely.

6 Discussion and Hypothesis Validation

- **H1 (Confirmed):** MTP speculative decoding provided a 2.5× speedup over Qwen-3.5-4B and 1.7× over Llama-3.2-3B on identical hardware.
- **H2 (Confirmed):** Quantization preserved JSON tool-calling fidelity, but arithmetic reasoning was inadequate across all sub-3B models without external compute tools.
- **H3 (Confirmed):** Qwen-3.5-4B consumed 3,069 MiB (74.9% of VRAM capacity). Deep contexts ($n_{ctx} > 16k$) require KV cache quantization to prevent OOM.
- **H4 (Confirmed with Nuance):** Direct safety alignment remained robust across 2B–4B parameter models, but universal vulnerability to indirect prompt injection demonstrates that parameter scale does not solve data/instruction boundary confusion.
- **H5 (Confirmed):** Multi-turn agent reliability is distinct from parameter size; Gemma-4-E2B handled tool chaining flawlessly, whereas Llama-3.2-3B failed.

7 Conclusions and Practical Recommendations

1. **Primary Edge Deployment Recommendation: Gemma-4-E2B-it** with MTP Q8_0 draft. Delivers > 40 tokens/s, consumes under 1.6 GB VRAM, and provides robust tool calling for agents like Pulse.
2. **Critical Semantic Reasoning Choice: Qwen-3.5-4B-Instruct** (Q4_K_M) when textual nuance takes precedence over generation speed.
3. **Hybrid Architecture Imperative:** Compact models must be paired with external deterministic calculation tools via MCP/Pulse to overcome intrinsic arithmetic limitations.
4. **Indirect Injection Guardrails:** Untrusted external strings (logs, web fetches, API responses) must be wrapped in strict cryptographic or structural boundary delimiters before reaching the SLM context.

Reproducibility and Open Artifacts

The full benchmark suite, structured JSON/CSV datasets, and replication instructions are open-source at:

https://github.com/AndersonTavares0/llm-research-bench